Relay: Complete Project Documentation

1. Executive Summary

Relay is an AI-powered developer command center and mobile companion. It is designed to act as a zero-latency bridge between a developer's local environment (IDE), their team, and their CI/CD cloud infrastructure. Instead of relying on fragmented dashboards and context-switching, Relay provides a unified, pitch-black mobile UI that surfaces only the most critical, actionable intelligence in real-time.

---

2. System Architecture

Relay operates on a modern, decoupled architecture designed for speed and telemetry.

2.1 The Mobile Client (Frontend)

- **Framework:** React Native managed by Expo.
- **UI System:** Material 3 (MD3) Design Language. Strictly uses OLED pitch-black (`#000000`) surfaces for battery efficiency and developer focus.
- **Animations:** Powered by `react-native-reanimated` (60fps native thread execution) for fluid micro-interactions, waveform breathing, and gesture handling.
- **Voice Capabilities:** Deep integration with ElevenLabs (Multilingual v2) for hyper-realistic Text-to-Speech (TTS), enabling the assistant to converse naturally.

2.2 The Telemetry Bridge (Backend & Integration)

Note: The mobile app is designed to interface with the following ecosystem:

- **Local Context:** A VS Code Extension that captures local IDE context (active files, branch, uncommitted diffs).
- **Relay Core (Backend):** A high-performance Go (Golang) server that ingests webhooks from GitHub/GitLab, CI/CD pipelines, and local IDEs, streaming them to the mobile app via Server-Sent Events (SSE).

---

3. Core Features & Views

3.1 The Home Dashboard (`HomeView.tsx`)

The entry point of the application, completely redesigned to eliminate "Dashboard Fatigue".

- **Hero Section:** A compact greeting with a sleek glowing action pill to instantly summon the AI.
- **Actionable Attention Cards:** Swipeable alerts (e.g., "TypeScript pipeline failed") that allow developers to retry pipelines or review PRs instantly.
- **Recent Activity:** A compressed timeline using strict iconography to differentiate between commits, PRs, and voice notes.
- **Interactive Team Dock:** A floating dock showing teammates' real-time presence and active files.

3.2 The Voice Assistant (`ProjectWorkspaceView.tsx`)

A completely custom, Apple-style minimalist capsule interface.

- **Audio Stack:** Utilizes `expo-audio` for recording and playback.
- **Wake-word & Speech Recognition:** Uses `expo-speech-recognition` to transcribe user commands.
- **ElevenLabs Integration:** Calls the ElevenLabs API (`eleven_multilingual_v2`) using a specific high-fidelity voice model to read out logs, summarize PRs, and answer queries.
- **Dynamic Waveforms:** Features real-time, react-native-reanimated audio waveforms that scale based on the assistant's state (Idle, Listening, Processing, Speaking).

3.3 Project Navigation (`ProjectsView.tsx` & `dashboard.tsx`)

- **Multi-Project Management:** Easily switch between different microservices or repositories (e.g., `relay-core`, `relay-app`, `relay-vscode`).
- **Floating Bottom Nav:** A custom-built, floating pill-shaped navigation bar that persists across screens.

---

4. Technical Stack & Dependencies

Dependency	Purpose
Expo SDK	Core framework, routing, and native module management.

React Native Reanimated	Complex, high-performance UI animations (Capsule expansion, waveforms).
React Native SVG	Custom iconography and vector graphics.
Expo Speech Recognition	On-device voice-to-text transcription.
Expo Audio	Advanced audio player and recorder for the AI voice.
ElevenLabs API	Next-generation Text-to-Speech generation.
Expo Haptics	Tactile feedback for premium physical interaction.

---

5. Design Philosophy (MD3 OLED)

The app strictly adheres to the `RELAY_MD3_DESIGN.md` guidelines:

- **No pure white backgrounds:** Relies on `#000000` (Pitch Black) and `#1C1C1E` (Surface Container Low).
- **Minimalist Branding:** The primary brand accent is a soft pink/rose, used sparingly for active AI states.
- **No Clutter:** If data is not immediately actionable, it is hidden.

---

6. Future Roadmap

1. **Push Notifications:** Integrating Expo Push Notifications for immediate alerts when pipelines fail while the app is in the background.
2. **WebSockets/SSE:** Connecting the frontend to the live Go backend for true real-time syncing.
3. **LLM Command Execution:** Passing voice transcripts through OpenAI to dynamically trigger GitHub Actions (e.g., "Relay, re-run the staging pipeline").